

PASOS PARA HACER UN DISEÑO MULTINIVEL EN FPGA - VIVADO

1. Creación del Proyecto:

Crear un nuevo proyecto eligiendo el nombre del proyecto y la carpeta que contendrá todos los archivos del diseño. Posteriormente elegir la familia, el dispositivo y el empaquetado de la FPGA que usará (Esta información figura en la caja del producto). Esto debe hacerse solo 1 vez.

FPGA de la placa de prueba basys 3

xc7a35t1cpg236-1L

2. Diseño de Bloques:

- a) Para agregar un nuevo debe seleccionar "Add Sources" y luego seleccionar "Add or Create design sources". Luego seleccionamos

Create file -> Seleccionar VHDL como lenguaje -> Colocar el nombre -> OK -> Finish.

Luego tendremos la opción de asignar puertos de entrada y salida (opcional).

Los bloques de RTL se crean definiendo las entradas y salidas, y en el código que prepara el sistema se puede codificar el funcionamiento escribiendo en el lenguaje de programación VHDL.

- b) Para agregar un nuevo bloque de diseño esquemático debemos ir a la sección que dice "Create Block Design" y luego le asignamos el nombre.

Los Diagramas en bloques que representan un diseño en esquemático. Se debe agregar un el símbolo de + el bloque llamado "Utility Vector Logic" si queremos agregar compuertas AND, OR, NOT o XOR. Existen algunos otros bloques que son propiedad intelectual (IP) del fabricante.

Otros dos bloques que son muy útiles pueden ser: "Concat" para unir varios bits en un BUS de datos o "Slice" para separar un BUS de datos en menos bits.

Para declarar una salida o una entrada hay que seleccionar y marcar donde dice "Make External"

Para agregar diseños de nuestra librería a un "Block Design" simplemente debemos crear un nuevo bloque de diseño, luego ir a la sección de:

Sources -> seleccionar con click derecho la fuente (puede ser diseñado por software o por esquemático) -> seleccionar la opción que dice "Add Module to Block Design". Con esto obtendremos una caja negra con las entradas y salidas del bloque que creamos.

3. Simulación de Bloques:

Para agregar un archivo de simulación nuevo debe seleccionar "Add Sources" y luego seleccionar "Add or Create simulation sources". Luego seleccionamos

Create file -> VHDL como lenguaje -> Colocar el nombre (aclarar que es un TB) -> OK -> Finish.

Una vez creado se deben seguir los siguientes pasos:

- 1) Por debajo de la línea que se crea por defecto "architecture Behavioral of Code_block_tb is" declaramos el componente que vamos a utilizar, que básicamente va a ser el bloque que queremos poner bajo prueba.

- 2) Declaramos las señales que vamos a utilizar, en este paso se deben crear señales para cada una de las variables que cuenta el componente instanciado, tanto de entrada como de salida.
- 3) Realizamos un mapeo a un bloque llamado Device Under Test (DUT) (el nombre es opcional, pero es recomendado). Lo que se debe hacer es una asignación de mapeo a las señales antes creadas, por ejemplo: `A_0 => A_0_s` donde "A_0" es la variable que contiene el componente original y "A_0_s" es la señal creada en el paso 2).
- 4) Iniciamos un process donde vamos asignarles valores a las señales de entrada creadas en el segundo paso. Luego de asignarle un valor se puede esperar unos nano segundos y volver a cambiarles el valor.

A continuación, se adjunta una imagen de un código que genera la simulación para una AND de 2 entradas y que prueba con todas las combinaciones posibles:

```
architecture Behavioral of AND_2a1_TB is
--Declaramos el componente que vamos a utilizar
component AND_2a1 is
    Port ( in_0 : in STD_LOGIC;

           in_1 : in STD_LOGIC;
           out_and : out STD_LOGIC);

end component;
--Declaramos las señales que vamos a utilizar
signal in_0_s: STD_LOGIC;
signal in_1_s: STD_LOGIC;
signal out_and_s: STD_LOGIC;
begin
-- Realizamos el mapeo del Device Under Test
DUT: AND_2a1_TB port map( in_0 => in_0_s, in_1 => in_1_s, out_and => out_and_s);
process
begin
--Debemos asignarle valores a las SEÑALES
in_0_s <= '0';
in_1_s <= '0';
wait for 10 ns;
in_0_s <= '0';
in_1_s <= '1';
wait for 10 ns;
in_0_s <= '1';
in_1_s <= '0';
wait for 10 ns;
in_0_s <= '1';
in_1_s <= '1';
wait;
end process;
end Behavioral;
```

Una vez creado un archivo como el que se observa en la imagen debemos hacer click derecho sobre el archivo creado y seleccionar la opción "Set as Top". Run simulation -> Run Behavioral Simulation.

Por último, si todo ha ido bien deberíamos tener las formas de onda con cada una de las señales.

4. RTL Analysis:

Se utiliza para analizar el comportamiento y la funcionalidad de un diseño de hardware descrito en lenguaje RTL (Register Transfer Level). Se debe Seleccionar “Run Linter” y esperar a que termine el proceso.

5. Synthesis:

Transforma el diseño RTL en una representación lógica optimizada y mapeada a los recursos del dispositivo de destino, preparándolo para la siguiente etapa de implementación física.

Debemos seleccionar “Run Synthesis”.

6. Implementation:

El proceso de implementación se basa en convertir el diseño sintetizado en un diseño físico listo para ser cargado y ejecutado en un dispositivo FPGA, asegurando que cumpla con los requisitos de rendimiento, temporización y funcionalidad especificados.

Seleccionamos “Run Implementation”, dejamos la carpeta por default y le clickeamos “OK”. Apenas termina seleccionamos en la opción que dice “Open implemented Design” y seleccionamos “OK”.

7. Asignación de Pines de la FPGA:

Dentro de la implementación, debemos asignar los pines de la FPGA que vamos a mapear a nuestras variables, utilizando los periféricos que a placa Basys 3 nos ofrece.

Se recomienda ir a la opción Layout -> I/O Planning. Con esto veremos una ventana con un mapa que tiene filas que son letras y columnas que se mapean con números como se muestra a continuación:



Luego teniendo como referencia el datasheet en la sección "I/O Ports" se debe mapear las variables a los componentes de prueba que necesitemos asignando valores en la columna "Package Pin".

Una recomendación importante es que se debe setear el nivel de tensión de las entradas y las salidas a una tensión que pueda trabajar, el cual, no es la que viene por defecto. Para ello, en la columna "I/O Std" se debe setear "LVCMOS33" en cada una de las variables que vamos a mapear.

Una vez terminada la asignación se debe repetir los procesos de Syntesis y de Implementation.

8. Generación de archivo de programación:

Una vez asignados los pines debemos darle click donde dice "Generate Bitstream".

Suele guardarse por defecto en:

Nombre_proyecto/ Nombre_proyecto.runs/impl_1/nombre_bloque.bit

Considerar que el "Nombre_proyecto" y "nombre_bloque" va a depender del nombre que tenga el proyecto que estemos trabajando.

9. Subir el archivo a la FPGA:

Luego de conectar la placa, debemos ir a la sección "Open Hardware Manager" luego en la parte superior clicar en Open Target -> Auto Connect. Una vez encontrado el dispositivo, debemos programarlo donde clickeando en la opción "Program Device".

Va a solicitar el archivo .bit generado en el paso anterior el cual debemos seleccionar y luego darle a "OK". Con esto, el programa queda correctamente grabado en la placa y podemos testear su funcionalidad.